

Proof of Deploy 1 Pager

Global AI Hackathon Series with Qwen Cloud

■ Hackathon Overview

Theme	Build AI Agents using Qwen Cloud models & APIs
Tracks	5 — MemoryAgent • AI Showrunner • Agent Society • Autopilot Agent • EdgeAgent
Team Size	Individuals, Teams, or Organizations
Development	May 26 – July 9, 2026
Final Submission	July 9, 2026 - 2 PM PT
Judging Period	July 10 – July 31, 2026
Results	August 7, 2026
IP Ownership	Participants retain full ownership

■ The Five Tracks

- **MemoryAgent:** Build agents with persistent, structured memory that learn and recall context across sessions.
- **AI Showrunner:** Create AI-driven content production pipelines — video, audio, scripts, or multimedia.
- **Agent Society:** Design multi-agent systems that collaborate, negotiate, or compete in shared environments.
- **Autopilot Agent:** Develop autonomous agents that handle real-world workflows end-to-end with minimal human input.
- **EdgeAgent:** Optimize AI agents for edge deployment — low latency, resource-constrained, on-device inference.

■ Judging Criteria (Weighted)

Criterion	Weight	What Judges Look For
Innovation & AI Creativity	30%	Is the architecture high-quality, with strong modularity, scalability, and error handling? Does the project reflect clean code and non-trivial logic? Does the tech stack show sophistication via advanced patterns and thoughtful adoption?

Technical Depth & Engineering	30%	Does the project make sophisticated use of QwenCloud APIs (e.g., custom skills, MCP integrations)? Does the project demonstrate algorithmic or engineering innovation through novel solutions, custom components, or performance optimization?
Problem Value & Impact	25%	Does the project have real-world relevance by solving an authentic technical or business pain point? Does the project have scalability potential for productization or open-source community adoption?
Presentation & Documentation	15%	Is the technical demo clear, with the key logic visualized effectively? Is the documentation clear, including architecture docs that describe the project?

■ Submission Checklist

- Provide a URL to your code repository for judging and testing. The repository must contain all necessary source code, assets, and instructions required for the project to be functional. The repository must be public and open source by including an open source license file. This license should be detectable and visible at the top of the repository page (in the About section).
- 1-3-minute video demo (max) showing the agent in action
- Architecture Diagram showing a clear visual representation of your system
- Written summary explaining the features and functionality
- Proof of Alibaba Cloud Deployment using one of the below sample screenshots demonstrating that the backend is running on Alibaba Cloud.
- Open-source frameworks OK; direct repo cloning is not allowed

■ Prize Structure (\$70,000+ Total)

Award	Count	Prize per Winner
Track Winners	5	\$10,000 each (\$7K cash + \$3K credits)
Best Blog Authors	10	\$1,000 each (\$500 cash + \$500 credits)
Honorable Mentions	10	\$1,000 each (\$500 cash + \$500 credits)

QUICKSTART — Base Code for Participants

Copy-paste these snippets to get started with Qwen Cloud APIs in minutes.

■ 1. Get Your API Key

Sign up at qwencloud.com, then generate a key at home.qwencloud.com/api-keys.

Set it as an environment variable (never hardcode secrets):

```
# Linux / macOS
export DASHSCOPE_API_KEY="sk-your-key-here"

# Windows PowerShell
$env:DASHSCOPE_API_KEY = "sk-your-key-here"
```

■ 2. Python — Chat Completion (OpenAI SDK Compatible)

Qwen Cloud is fully compatible with the OpenAI SDK. Install with: `pip install openai`

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url="https://dashscope-intl.aliyuncs.com/compatible-mode/v1"
)

completion = client.chat.completions.create(
    model="qwen3.7-plus",
    messages=[
        {"role": "user", "content": "Summarize the benefits of solar energy."}
    ]
)

print(completion.choices[0].message.content)
```

■ 3. Python — Streaming Responses

```
stream = client.chat.completions.create(
    model="qwen3.7-max",
    messages=[
        {"role": "user", "content": "Write a haiku about AI agents."}
    ],
    stream=True
)
```

```

for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="")

```

■ 4. Python — Function Calling (Tool Use)

```

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name"}
            },
            "required": ["city"]
        }
    }
}]

response = client.chat.completions.create(
    model="qwen3.7-plus",
    messages=[{"role": "user", "content": "What's the weather in Jakarta?"}],
    tools=tools
)

print(response.choices[0].message.tool_calls)

```

■ 5. Python — Image Generation (Wan2.6)

```

response = client.images.generate(
    model="wan2.6-t2i",
    prompt="A futuristic city skyline at sunset with flying cars",
    size="1024x1024",
    n=1
)

print(response.data[0].url)

```

■ 6. Python — Text-to-Speech (CosyVoice)

```

import requests

resp = requests.post(
    "https://dashscope-intl.aliyuncs.com/api/v1/services/aigc/text2audio/generation",
    headers={
        "Authorization": f"Bearer {os.getenv('DASHSCOPE_API_KEY')}",
        "Content-Type": "application/json"
    }
)

```

```

    },
    json={
      "model": "cosyvoice-v3-plus",
      "input": {"text": "Hello, welcome to the Qwen Cloud Hackathon!"},
      "parameters": {"voice": "default"}
    }
  )

```

API Reference & Model Catalog

■ API Endpoints (Base URL: dashscope-intl.aliyuncs.com)

Endpoint	Description
POST /compatible-mode/v1/chat/completions	OpenAI-compatible chat completions (text & vision)
POST /compatible-mode/v1/responses	OpenAI Responses API (multi-step agent workflows)
POST /apps/anthropic/v1/messages	Anthropic-compatible messages endpoint
POST /api/v1/.../text-generation/generation	DashScope native text generation
POST /services/aigc/multimodal-generation/generation	Image generation (sync)
POST /services/aigc/text2image/image-synthesis	Image generation (async, poll with task ID)
GET /tasks/{task_id}	Poll async task status

■ Available Models

Model ID	Type	Best For
qwen3.7-max	Text + Reasoning	Complex tasks
qwen3.7-plus	Text + Vision	Balanced perf.
qwen3.6-flash	Text	Fast & cheap
happyhorse-1.1-t2v	Text → Video	Video generation
happyhorse-1.1-i2v	Image → Video	Video from images

wan2.6-t2i	Text → Image	Image generation
qwen-image-2.0-pro	Image gen + edit	Creative assets
cosyvoice-v3-plus	Text → Speech	Voice / TTS

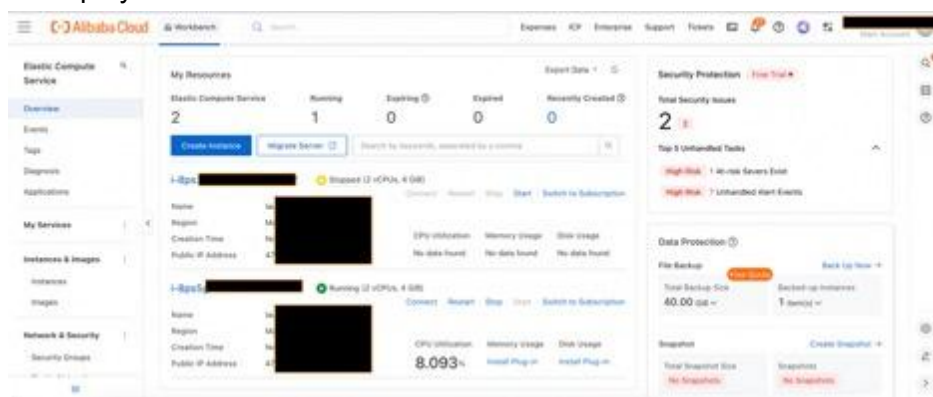
Free tier: 70M+ tokens available for all new accounts.

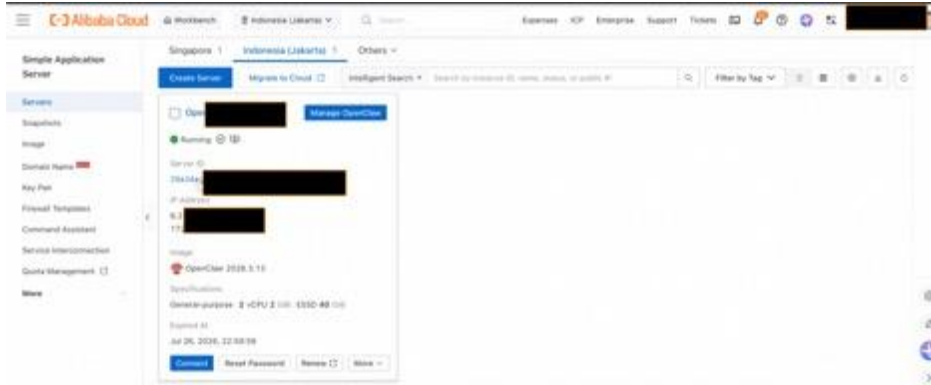
■ Key Links for Participants

Resource	URL
Hackathon Page	qwencloud.com/challenge/hackathon
Documentation	docs.qwencloud.com
API Key Portal	home.qwencloud.com/api-keys
Model Catalog	qwencloud.com/models
Interactive Playground	home.qwencloud.com/try-ai
GitHub	github.com/QwenCloud
Devpost (Submissions)	devpost.com (search Qwen Cloud)
Discord Community	Linked from hackathon page

Sample Screenshot

Find your Workbench Overview and take a screenshot of the running project to show proof of Alibaba Cloud Deployment.





Token Plan vs. Pay-as-you-go

	Pay-as-you-go	Token Plan
API Key Prefix	sk-xxxxxx	sk-sp-xxxxxx
Base URL	dashscope-intl.aliyuncs.com/compatible-mode/v1	token-plan.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1
Billing	Per-token, monthly invoice	Credit pool deducted per request
Rate Limits	Standard	Higher limits
Usage Scope	All use cases	Interactive dev tools (no backend scripts)

Important: Do NOT mix Token Plan keys with pay-as-you-go endpoints (or vice versa). Doing so returns 401 InvalidApiKey.

■ Token Plan Base URLs

Protocol	Base URL
OpenAI Compatible	https://token-plan.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1
Anthropic Compatible	https://token-plan.ap-southeast-1.maas.aliyuncs.com/apps/anthropic

■ Token Plan — Python (OpenAI SDK)

Install: `pip install openai`

```
import os
from openai import OpenAI
```

```

client = OpenAI(
    api_key=os.getenv("DASHSCOPE_API_KEY"), # sk-sp-xxxxx

base_url="https://token-plan.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1"
)

completion = client.chat.completions.create(
    model="qwen3.7-plus",
    messages=[
        {"role": "user", "content": "Build me an AI agent that summarizes
documents."}
    ]
)

print(completion.choices[0].message.content)

```

■ Token Plan — Node.js (OpenAI SDK)

Install: `npm install openai`

```

import OpenAI from "openai";

const openai = new OpenAI({
    apiKey: process.env.DASHSCOPE_API_KEY, // sk-sp-xxxxx
    baseURL:
"https://token-plan.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1"
});

const completion = await openai.chat.completions.create({
    model: "qwen3.7-plus",
    messages: [
        { role: "user", content: "Build me an AI agent that summarizes
documents." }
    ]
});

console.log(completion.choices[0].message.content);

```

■ Token Plan — cURL

```

curl -X POST
https://token-plan.ap-southeast-1.maas.aliyuncs.com/compatible-mode/v1/chat/completions \
-H "Authorization: Bearer $DASHSCOPE_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.7-plus",
  "messages": [
    { "role": "user", "content": "Build me an AI agent." }
  ]
}'

```


How To Deploy on Alibaba Cloud

Create the ECS Instance

Via the Console

1. Log in to the [ECS Console](#) and click **Create Instance**.
2. Select **Custom Launch** for full control.
3. Choose a **billing method**
4. Select **Region & Zone**.
5. Pick your **Instance Type** from the families listed above.
6. Choose an **Image** — Ubuntu 22.04 or 24.04 LTS is recommended for AI workloads.
7. Configure **Storage**
8. Set up **Networking**.
9. Attach a **Security Group**.
10. Set **Login Credentials** — use a Key Pair (recommended over passwords).
11. Under **Advanced**, set a descriptive instance name, optionally attach a RAM role, and add User Data scripts for automated bootstrapping.
12. Review, accept the Terms of Service, and click **Create Order**.

Set Up Networking

VPC: Create a VPC with a private CIDR block (e.g., 172.16.0.0/12). Create VSwitches across multiple zones if you plan to use Auto Scaling or load balancing later.

Public IP: Either assign a public IPv4 address directly (pay-by-bandwidth or pay-by-traffic) or attach an Elastic IP (EIP) for flexible association. For backend instances that don't serve internet traffic directly, skip the public IP and use a NAT Gateway for outbound access.

Security Groups: These act as whitelists (deny-all by default). Open only the ports your agent needs:

Port	Protocol	Source	Purpose
22	TCP	Your IP only	SSH access
80	TCP	0.0.0.0/0	HTTP (if serving web traffic)
443	TCP	0.0.0.0/0	HTTPS
8080	TCP	Your IP or LB	Agent API (if applicable)

Never open SSH (22) or RDP to 0.0.0.0/0 in production.

Connect to the Instance

Method	Best for
Workbench (browser-based)	Quick access, no local software needed
SSH (OpenSSH / PuTTY)	Standard terminal access
VNC Console	Troubleshooting when SSH fails
Session Manager	Auditable access without a public IP

```
# SSH with key pair
ssh -i ~/.ssh/my-alibaba-key.pem root@<public-ip>

# Harden: disable password auth
sudo sed -i 's/^PasswordAuthentication yes/PasswordAuthentication no/'
/etc/ssh/sshd_config
sudo systemctl restart sshd
```

Install Dependencies

Docker (recommended)

```
# Add Docker repo
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker.gpg
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker-compose-plugin

# Start and enable
sudo systemctl start docker && sudo systemctl enable docker

# Allow non-root Docker usage
sudo usermod -aG docker $USER
```

Python (bare-metal)

```
sudo apt-get update && sudo apt-get install -y python3 python3-pip python3-venv git

# Optional: Miniconda for AI workloads
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

Node.js

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Deploy the Agent

Option A: Docker Compose (multi-service)

This is the recommended approach for agents that need supporting services (database, cache, reverse proxy).

```
# docker-compose.yml
version: '3.8'
services:
  agent:
    build: .
    ports:
      - "8080:8080"
    environment:
      - API_KEY=${API_KEY}
      - LOG_LEVEL=info
    volumes:
      - ./data:/app/data
    restart: unless-stopped
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./certs:/etc/nginx/certs
    restart: unless-stopped
    depends_on:
      - agent

docker compose up -d
```

Option B: Single Container

```
docker run -d \
  --name ai-agent \
  --restart unless-stopped \
  -p 8080:8080 \
  --env-file .env \
```

```
-v /opt/agent/data:/app/data \
my-ai-agent:latest

# With GPU support:
docker run -d --gpus all --name ai-agent -p 8080:8080 my-ai-agent:latest
```

Option C: Bare-Metal with systemd

```
# Clone and set up
git clone https://github.com/your-org/ai-agent.git /opt/ai-agent
cd /opt/ai-agent
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Create a systemd service for persistence:

```
# /etc/systemd/system/ai-agent.service
[Unit]
Description=AI Agent Application
After=network.target

[Service]
Type=simple
User=agent
WorkingDirectory=/opt/ai-agent
ExecStart=/opt/ai-agent/venv/bin/python main.py
Restart=always
RestartSec=10
Environment=NODE_ENV=production
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable ai-agent
sudo systemctl start ai-agent
sudo systemctl status ai-agent
```

Deploying an Agent on Alibaba Cloud SAS

When to Use SAS Instead of ECS

Simple Application Server (SAS/SWAS) bundles compute, storage, and networking into predictable monthly plans. It trades flexibility for simplicity.

Choose SAS if...	Choose ECS instead if...
You need a single agent server for dev/test or low-traffic production	You need GPU instances for local model inference
You want fixed, predictable pricing with no surprise bills	You need Auto Scaling or load balancing
You want to deploy in under 5 minutes	You need full VPC control or VPC peering
Your agent calls external LLM APIs (no local GPU needed)	Your workload is high-concurrency or mission-critical
You're an individual developer, student, or small team	You need multiple data disks or specialized storage

Best-fit agent types for SAS: LLM API wrapper agents, Dify / MaxKB deployments, lightweight RAG pipelines, chatbot backends, webhook-based automation agents.

Create the Instance

1. Go to the [SAS Console](#).
2. Click **Create Server**.
3. Select a **Region**.
4. Select an **Image**.
5. Select your **Plan**
6. Choose **quantity** and **subscription duration**.
7. Complete payment. The instance is provisioned immediately with an auto-assigned public IP.

Important: SAS instances have no default password. Before connecting, reset the root password from the console under **Reset Password**.

Choose an Image

Application Images (pre-installed software — fastest path)

These come with a ready-to-use software stack on top of Alibaba Cloud Linux 3:

Image	Best for
Dify v0.15.3	LLM app platform deployment
MaxKB v1.10.2	Knowledge-base chatbot
Docker v26.1.3	Custom containerized agent
LAMP / LNMP	Web-based agent dashboards
BT-Panel / 1Panel	Non-technical server management

For a custom agent, the Docker image is the recommended starting point.

Connect to the Instance

Linux

Method	How
Workbench (recommended)	Click "Connect" on the instance card in the console. Logs in as root via internal auth — no password needed.
SSH	Use any SSH client. Connect as `root` with the password you set in Step 3.
Rescue Mode (VNC)	Use when SSH fails (e.g., instance under heavy load). Console > Connect > Rescue. Auto-disconnects after ~5 min inactivity.

```
# SSH from your terminal
ssh root@<public-ip>
```

Windows

Use **Workbench** (RDP via browser) or **Remote Desktop** (mstsc) with the Administrator password you set in the console.

Configure the Firewall

SAS has a built-in firewall that controls inbound traffic only (outbound is unrestricted). It supports up to 50 rules per instance.

Default rules (Linux): TCP 22, 80, 443 + ICMP.

Open the ports your agent needs:

Port	Protocol	Source	Purpose
22	TCP	Your IP range only	SSH (restrict source)
80	TCP	0.0.0.0/0	HTTP
443	TCP	0.0.0.0/0	HTTPS
8080	TCP	0.0.0.0/0 or restricted	Agent API endpoint

To add a rule: SAS Console > your instance > **Firewall** tab > **Add Rule** > select a preset (HTTP, SSH, MySQL, etc.) or define a custom port range.

Security tip: Restrict SSH (port 22) to your known IP ranges rather than leaving it open to all sources.

Install Dependencies

Option A: Docker Application Image (recommended)

If you selected the Docker image at creation, Docker Engine and Docker Compose are already installed. Verify:

```
docker --version          # Docker version 26.1.3
docker compose version    # Docker Compose version 2.x
```

Skip ahead to Section 8.

Option B: Manual Install on a Clean OS Image

Docker (Ubuntu):

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y ca-certificates curl gnupg

# Add Docker's official GPG key and repo
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```



```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker-compose-plugin
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker $USER
```

Python (bare-metal):

```
sudo apt-get update && sudo apt-get install -y python3 python3-pip python3-venv git
curl
```

Node.js:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Deploy the Agent

Option A: Docker Compose (recommended)

Best for agents with supporting services (database, cache, reverse proxy).

```
# docker-compose.yml
version: '3.8'
services:
  agent:
    build: .
    ports:
      - "8080:8080"
    environment:
      - API_KEY=${API_KEY}
      - LOG_LEVEL=info
    volumes:
      - ./data:/app/data
    restart: unless-stopped
    mem_limit: 3g          # Leave headroom for the OS

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./certs:/etc/nginx/certs
    restart: unless-stopped
```

```
    depends_on:
      - agent

# Create a .env file with your secrets
echo "API_KEY=your-api-key-here" > .env

# Start the stack
docker compose up -d

# Verify
docker compose ps
docker compose logs -f agent
```

Option B: Single Container

```
docker run -d \
  --name ai-agent \
  --restart unless-stopped \
  --memory 3g \
  -p 8080:8080 \
  --env-file .env \
  -v /opt/agent/data:/app/data \
  my-ai-agent:latest
```

Option C: Bare-Metal with Process Manager

```
# Clone and set up
git clone https://github.com/your-org/ai-agent.git /opt/ai-agent
cd /opt/ai-agent
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# Option 1: systemd service
sudo tee /etc/systemd/system/ai-agent.service << 'EOF'
[Unit]
Description=AI Agent Application
After=network.target

[Service]
Type=simple
User=agent
WorkingDirectory=/opt/ai-agent
ExecStart=/opt/ai-agent/venv/bin/python main.py
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable ai-agent && sudo systemctl start ai-agent
```

```
# Option 2: PM2 (Node.js agents)
npm install -g pm2
pm2 start main.js --name ai-agent
pm2 save && pm2 startup
```

Deploying Dify (Quick Start with Application Image)

If you chose the **Dify** application image at creation, the platform is already installed with Ollama and sample models. Here's how to get it running:

- Connect via Workbench or SSH.
- Check the Dify service status: `docker compose ps` (in the Dify project directory).
- Open port 80 (or the configured port) in the firewall.
- Navigate to `http://<public-ip>` in your browser.
- Create an admin account on first visit.
- Configure your LLM API keys (Qwen Cloud, etc.) in Settings > Model Provider.